

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2020 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2020	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Liste dublu înlănțuite	2
2. Rețele de calculatoare: Topologii și medii de transmisie	2
II. Rezolvare Titularizare Varianta 3 (29 iulie 2020)	2
SUBIECTUL I	2
1. Algoritmul de sortare prin Interclasare (Merge Sort)	2
2. Procesoare de pagini web: Concepte HTML	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Șiruri consonatice perechi	3
2. Algoritm eficient: Cel mai lung subșir crescător de termeni pari (LIS $O(N \log N)$) ..	6
SUBIECTUL al III-lea (30 de puncte)	7
1. Expunerea sistematică pentru matricea drumurilor	7
2. Observarea sistematică pentru comportamentul în Internet	7

I. Rezolvare Titularizare Model 2020

SUBIECTUL I (30 de puncte)

1. Structura de date: Liste dublu înălțuite

- **Definire:** O listă dublu înălțuită constă în noduri alocate dinamic, fiecare nod reținând valoarea informației proprii, adresa nodului următor (urm) și adresa nodului precedent (prec).
- **Operații:** Inserare la început/sfârșit/interior, ștergere nod specific, parcurgere în ambele direcții.
- **Problemă (Gestiune coadă de așteptare):**
 - **C++:**

```
#include <iostream>
using namespace std;
struct Nod {
    int info;
    Nod *prec, *urm;
};
void inserareSfarsit(Nod* &p, Nod* &u, int x) {
    Nod* nou = new Nod{x, u, nullptr};
    if (p == nullptr) { p = nou; u = nou; }
    else { u->urm = nou; u = nou; }
}
int main() {
    Nod *p = nullptr, *u = nullptr;
    inserareSfarsit(p, u, 10);
    inserareSfarsit(p, u, 20);
    for(Nod* c = p; c != nullptr; c = c->urm) cout << c->info << " ";
    return 0;
}
```

2. Rețele de calculatoare: Topologii și medii de transmisie

- **Topologii de rețea:** Stea (centralizată cu switch), Inel, Magistrală (Bus).
- **Medii fizice:** Cablu coaxial, cablu UTP/FTP (perechi răsucite), fibră optică (viteză mare pe distanțe lungi), transmisii wireless (Wi-Fi, Bluetooth).

II. Rezolvare Titularizare Varianta 3 (29 iulie 2020)

[Subiect PDF](file:

SUBIECTUL I

1. Algoritmul de sortare prin Interclasare (Merge Sort)

- **Descriere:** Merge Sort folosește tehnica Divide et Impera. Împarte vectorul în două jumătăți egale (sau aproximativ egale), le sortează recursiv, iar apoi interclasează cele două jumătăți sortate pentru a obține vectorul complet ordonat.
- **Trace pentru 7 numere:** [38, 27, 43, 3, 9, 82, 10]
 - Divide: [38, 27, 43, 3] și [9, 82, 10]
 - Divide mai departe până la elemente individuale, apoi le combină interclasează ordonat:
 - [38, 27] -> [27, 38]

- [43, 3] -> [3, 43]
- Combina: [27, 38] cu [3, 43] -> [3, 27, 38, 43]
- În mod similar pentru a doua parte: [9, 10, 82]
- Interclasare finală: [3, 27, 38, 43] cu [9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82].
- **Complexitate:** $O(N \log N)$ în toate cazurile (cel mai bun, cel mai rău, mediu).
- **Problemă (Merge Sort):**

```
#include <iostream>
using namespace std;
void merge(int A[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[50], R[50];
    for (int i = 0; i < n1; ++i) L[i] = A[l + i];
    for (int j = 0; j < n2; ++j) R[j] = A[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) A[k++] = L[i++];
        else A[k++] = R[j++];
    }
    while (i < n1) A[k++] = L[i++];
    while (j < n2) A[k++] = R[j++];
}
void mergeSort(int A[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(A, l, m);
        mergeSort(A, m + 1, r);
        merge(A, l, m, r);
    }
}
int main() {
    int A[] = {38, 27, 43, 3, 9, 82, 10};
    mergeSort(A, 0, 6);
    for (int x : A) cout << x << " ";
    return 0;
}
```

2. Procesoare de pagini web: Concepte HTML

- **Noțiuni:** Pagină web (fișier text ce conține cod de marcarea HTML), legături hipertext, publicarea pe servere web utilizând protocoale de tip HTTP/FTP.
- **Etichete:**
 - <p>: Text formatat ca paragraf (atribute: class, style).
 - : Inserare imagine (atribute: src, alt).
 - <table>: Creare tabel (atribute: border, cellpadding).
 - <a>: Legătură hipertext (atribute: href, target).

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Șiruri consonatice perechi

Soluție C++:

```
#include <iostream>
#include <string>
```

```
#include <sstream>
#include <vector>
#include <map>
using namespace std;

void consonatic(const string &s, string &sc) {
    sc = "";
    for (char c : s) {
        if (c != 'a' && c != 'e' && c != 'i' && c != 'o' && c != 'u') {
            sc += c;
        }
    }
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string cuvant;
    map<string, int> frecventa;

    while (ss >> cuvant) {
        string sc;
        consonatic(cuvant, sc);
        frecventa[sc]++;
    }

    long long perechi = 0;
    for (auto &pair : frecventa) {
        long long count = pair.second;
        perechi += count * (count - 1) / 2;
    }

    cout << perechi << endl;
    return 0;
}
```

Soluție Pascal:

```
program PerechiConsonantice;
var
    text_input, cuvant, sc: string;
    cuvinte, consoane: array[1..100] of string;
    nr_cuvinte, i, j, perechi: integer;
    frecv: array[1..100] of integer;
    deja_numarat: array[1..100] of boolean;

procedure consonatic(s: string; var sc: string);
var
    idx: integer;
begin
    sc := '';
    for idx := 1 to length(s) do
        begin
            if (s[idx] <> 'a') and (s[idx] <> 'e') and (s[idx] <> 'i') and (s[idx] <> 'o')
            and (s[idx] <> 'u') then
```

```

        sc := sc + s[idx];
    end;
end;

begin
    readln(text_input);
    text_input := text_input + ' ';
    nr_cuvinte := 0;
    cuvant := '';
    for i := 1 to length(text_input) do
    begin
        if text_input[i] <> ' ' then
            cuvant := cuvant + text_input[i]
        else
            begin
                if length(cuvant) > 0 then
                    begin
                        nr_cuvinte := nr_cuvinte + 1;
                        cuvinte[nr_cuvinte] := cuvant;
                        consonatic(cuvant, consoane[nr_cuvinte]);
                        cuvant := '';
                    end;
                end;
            end;
        end;

    for i := 1 to nr_cuvinte do
    begin
        frecv[i] := 0;
        deja_numarat[i] := false;
    end;

    perechi := 0;
    for i := 1 to nr_cuvinte do
    begin
        if not deja_numarat[i] then
            begin
                frecv[i] := 1;
                for j := i + 1 to nr_cuvinte do
                begin
                    if consoane[i] = consoane[j] then
                        begin
                            frecv[i] := frecv[i] + 1;
                            deja_numarat[j] := true;
                        end;
                    end;
                end;
                perechi := perechi + (frecv[i] * (frecv[i] - 1)) div 2;
            end;
        end;
    end;

    writeln(perechi);
end.

```

2. Algoritm eficient: Cel mai lung subșir crescător de termeni pari (LIS $O(N \log N)$)

Metoda: Prelucrăm numai termenii pari, deoarece subșirul cerut trebuie să conțină exclusiv valori pare. Folosim algoritmul eficient pentru cel mai lung subșir strict crescător. Vectorul `tail[k]` reține cea mai mică valoare posibilă a ultimului element al unui subșir crescător de lungime k . Pentru fiecare termen par x , căutăm binar prima poziție unde `tail[pos] >= x`; dacă există, înlocuim acea valoare cu x , altfel extindem vectorul. Dimensiunea finală a vectorului `tail` este lungimea maximă cerută.

Justificarea eficienței: Fiecare număr din fișier este citit o singură dată. Pentru fiecare termen par se execută o căutare binară în vectorul `tail`, deci complexitatea este $O(n \log n)$ în cel mai rău caz, cu memorie $O(n)$ pentru valorile intermediare. Este mai eficient decât programarea dinamică clasică $O(n^2)$.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2020.in");
    int x;
    vector<int> tail;
    while (fin >> x) {
        if (x % 2 == 0) {
            auto it = lower_bound(tail.begin(), tail.end(), x);
            if (it == tail.end()) {
                tail.push_back(x);
            } else {
                *it = x;
            }
        }
    }
    fin.close();

    cout << tail.size() << endl;
    return 0;
}
```

Soluție Pascal:

```
program LisePari;
var
    fin: text;
    tail: array[1..1000000] of longint;
    n_tail, x, st, dr, mid, pos: longint;
begin
    assign(fin, 'titu2020.in');
    reset(fin);
    n_tail := 0;

    while not eof(fin) do
        begin
            read(fin, x);
```

```

if x mod 2 = 0 then
begin
  st := 1;
  dr := n_tail;
  pos := -1;
  while st <= dr do
  begin
    mid := (st + dr) div 2;
    if tail[mid] >= x then
    begin
      pos := mid;
      dr := mid - 1;
    end
    else
      st := mid + 1;
    end;
  end;

  if pos = -1 then
  begin
    n_tail := n_tail + 1;
    tail[n_tail] := x;
  end
  else
    tail[pos] := x;
  end;
end;
close(fin);

writeln(n_tail);
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Expunerea sistematică pentru matricea drumurilor

Caracteristici: prezentare logică și graduală; utilizarea unui exemplu demonstrativ. **Avantaj:** algoritmi de grafuri cer introducerea riguroasă a terminologiei și a pașilor.

Mijloc: tablă/proiector cu matrice de adiacență. **Formă:** frontal. **Activitate:** construirea matricei drumurilor pentru un graf orientat.

Scenariu: Profesorul definește matricea, arată actualizarea prin nod intermediar, elevii completează câteva celule și justifică existența drumurilor.

2. Observarea sistematică pentru comportamentul în Internet

Caracteristici: urmărește comportamente reale; se realizează pe o perioadă de timp. **Instrument:** fișă de observație.

Comportamente temperamentale: respectă regulile de comunicare, reacționează calm la opinii diferite, manifestă răbdare în lucru colaborativ. **Comportamente cognitive:** recunoaște mesaje nepoliticoase/nesigure, aplică reguli de citare și respectare a legislației.